DAY 4 | HANDS-ON ACTIVITIES

Architecture and MongoDB: Problem Statements

Eight problems. Colab users: use mongomock as in the notebook. VS Code users: use a local MongoDB or a free Atlas cluster.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Expense Chart

Level: Warm-up | **Concepts:** Matplotlib bar chart

Given categories ["Food", "Rent", "Travel", "Fun"] and amounts [6000, 15000, 2500, 3000], draw a bar chart with a title, axis labels and the value on top of each bar.

Sample run / expected result

```
A bar chart titled "Monthly expenses" with 4 labelled bars
```

Hints

- ax.bar_label(bars) places the values.

Problem 2: Architecture Sketch

Level: Easy | **Concepts:** frontend, backend, middleware, layers

For a telecom support app, write down the clients, at least 5 API endpoints, the middleware checks, the server layers and the collections. No code needed.

Sample run / expected result

```
Clients: customer app, agent web console
Endpoints: GET /tickets, POST /tickets, ...
Middleware: logging, auth
Collections: customers, tickets, agents
```

Hints

- Think about what each screen in the app needs.

Problem 3: Ticket Catalogue: Create and Read

Level: Easy | **Concepts:** insert_many, find, filters, projection

Insert 20 sample tickets with customer, issue, priority, status and created date. Print all open tickets, all high-priority tickets, and the count of tickets per status.

Sample run / expected result

```
Open: 14
High: Mei, Kenji, Priya ...
By status: {'open': 14, 'closed': 6}
```

Hints

- {"priority": {"$in": ["high"]}}
- count_documents(filter)

Problem 4: Update and Delete Safely

Level: Medium | **Concepts:** update_one, update_many, $set, $push, delete

Close every ticket older than 7 days, add a "stale" tag to them with $push, then delete tickets marked as test. Print the filter's matches with find before each change.

Sample run / expected result

```
Will close: 5
Closed: 5
Will delete: ['TEST-1']
Deleted: 1
```

Hints

- Always run find(filter) before update_many or delete_many.

Problem 5: Indexes

Level: Medium | **Concepts:** create_index, unique index

Create an index on status, a compound index on (status, priority) and a unique index on ticket_no. Show that inserting a duplicate ticket_no raises DuplicateKeyError.

Sample run / expected result

```
Indexes: ['_id_', 'status_1', 'status_1_priority_-1', 'ticket_no_1']
DuplicateKeyError: ticket_no T-001
```

Hints

- from pymongo.errors import DuplicateKeyError

Problem 6: Embed or Reference?

Level: Medium | **Concepts:** data modelling

Design documents for SmartDesk customers, tickets, ticket notes and agents. For each relationship, decide embed or reference and give a one-line reason. Then insert one example of each.

Sample run / expected result

```
ticket -> notes     : embed (read together, small)
ticket -> customer  : reference (shared by many tickets)
```

Hints

- Ask: read together? shared? grows without limit?

Problem 7: SmartDesk: Chart by Priority

Level: Challenging | **Concepts:** MongoDB read + Matplotlib

SmartDesk step 3. Store tickets in a tickets collection. Count open tickets per priority in Python and draw a bar chart. Save it as tickets_by_priority.png.

Sample run / expected result

```
{'high': 5, 'medium': 6, 'low': 3}
tickets_by_priority.png saved
```

Hints

- Accumulate counts in a dict, then plt.bar(keys, values).

Problem 8: Repository Class

Level: Stretch | **Concepts:** OOP + MongoDB, layered design

Write a TicketRepository with add, get, list_by_status, close and delete. The rest of your code should only call the repository, never the collection directly.

Sample run / expected result

```
repo = TicketRepository(db["tickets"])
repo.add({...})
repo.list_by_status("open")
```

Hints

- Pass the collection into __init__ so it can be swapped for a real database later.